In-Memory Databases

Memory-optimized tables and natively compiled stored procedures in SQL Server

Vassányi, István
University of Pannonia, Veszprém

Overview

- Traditional OLTP relational architecture
  - Limited buffer cache
  - „Transactional” tables are typically fetched from secondary storage
- Memory getting cheaper, faster than table sizes grow
- Higher transaction throughput needed
- New architecture, new engine was designed
  - Memory Optimized Tables (MOT) are only written to storage at recovery or startup
  - More efficient in-memory data structures, natively compiled data processing
  - Optimized for fast transaction processing with lots of limitations
  - Integrated „seamlessly” with the less critical part of the database that does not use memory optimized technology
- In-memory is not fast because the table is in memory

Features

- Memory Optimized Tables
  - Hash indexes for primary keys (no range searches), 8-byte memory pointers
  - No clustered index. All nonclustered indexes can access all columns
  - JOINs are always hash joins
  - MOT can be non-durable (cache table), but the structure is persisted
  - „Transactional” tables are typically fetched from secondary storage
- Full ACID support
  - All changes are written to the on-disk transaction log
  - Automatic („optimistic”) snapshot isolation with row versioning
  - Tempdb is not used
  - No other isolation level supported
- Native stored procedures
  - For each stored procedure, a new dll is compiled

Scenarios

- Support for a large number of concurrent, low-latency transactions
  - Stock exchange or betting
- Real-time data acquisition and data transform from many sources
  - IoT sensor networks, old data can be swapped to an on-disk column store
- ETL using non-durable memory-optimized tables as staging tables
- Caching
  - non-durable MOT used as key-value store (blob or json)

DEMO: inserting

Limitations

- More memory needed for MOTs
  - Data, indexes, row versioning
- No cross-database queries
- No page/row compression, no DDL triggers
- Only transactional replication, subscriber role
- etc. …
- Native procs can use only MOTs
  - Lots of other limitations like no transaction management, no one-part names, no options setting, no LIKE, no print, no @@identity, no …, no …

Migration

- Database level settings
  - New filegroup and file to be created
  - Enabling automatic snapshot isolation
- Selecting the critical part of the database (tables and procs)
  - Built-in tools (Database -> Reports -> Standard reports -> Transaction Performance Analysis Overview)
- Perform migration
  - MOT: Drop foreign key constraints
  - MOT: create new table, insert … select from old (built-in tools)
  - Procs: rewrite manually observing the limitations
- „In-memory is not a magical go-fast button”

DEMO: order process